

Darío Alejandro Guzmán Pulgarín

Nombre:

Desarrollo de Software

Carrera:

3°A

Curso:

HERRAMIENTAS INFORMATICAS PARA EL DESPLIEGUE DE DIAGRAMAS

Clase:

Base De Datos Local en Excel

Actividad:

10/10/2026

Fecha:

INFORME DE TRAZABILIDAD DEL USO DE INTELIGENCIA ARTIFICIAL EN EL DESARROLLO DE UNA API REST CON NESTJS Y SQLITE

Asignatura: Herramientas informáticas para el despliegue de diagramas
Carrera: Tecnología en Desarrollo de Software
Ciclo: Tercero
Período académico: Septiembre 2026 - febrero 2027
Estudiante: Darío Alejandro Guzmán Pulgarín
Herramienta de inteligencia artificial: ChatGPT

1. Introducción

El presente informe documenta el proceso de utilización de inteligencia artificial como herramienta de apoyo para desarrollar una aplicación backend utilizando NestJS, tomando como referencia el material de estudio *Estructura HTTP y JSON* y los resultados de aprendizaje establecidos en el programa de la asignatura.

El proyecto consiste en implementar una API REST para gestionar registros de usuarios mediante operaciones de creación, consulta, actualización y eliminación. Para almacenar la información se planteó utilizar SQLite, una base de datos que guarda los registros en un archivo local dentro del proyecto, sin depender de una aplicación independiente para administrar un servidor de base de datos.

Durante el proceso se utilizó ChatGPT para obtener orientación técnica, comprender la organización del proyecto, solucionar errores de configuración y aprender a probar los endpoints mediante Postman. Las consultas se realizaron progresivamente, de acuerdo con las dificultades encontradas durante la implementación.

2. Objetivos

2.1. Objetivo general

Documentar el proceso de utilización de ChatGPT en el desarrollo de una API REST con NestJS y SQLite, identificando las consultas realizadas, los resultados obtenidos, las dificultades técnicas y los mecanismos de verificación necesarios.

2.2. Objetivos específicos

- Identificar los prompts utilizados durante las diferentes etapas del desarrollo.
- Explicar cómo las respuestas de la inteligencia artificial contribuyeron a la construcción de la aplicación.
- Relacionar el proyecto con los contenidos de HTTP, JSON, API REST y persistencia de datos.
- Registrar los errores encontrados y las acciones propuestas para solucionarlos.
- Promover el uso ético y responsable de la inteligencia artificial mediante la revisión y comprobación del código generado.

3. Descripción del proyecto

La aplicación se desarrolló utilizando NestJS como framework backend. Su finalidad es gestionar usuarios a través de endpoints que reciben solicitudes HTTP y devuelven respuestas en formato JSON.

La estructura propuesta incluye módulos, controladores, servicios, entidades y objetos de transferencia de datos (DTO), con el propósito de separar las responsabilidades del sistema.

La persistencia de la información se planteó mediante SQLite y TypeORM. Los registros se almacenan en un archivo local denominado `datos.sqlite`.

Las principales operaciones previstas son:

- Crear usuarios mediante POST.
- Consultar todos los usuarios mediante GET.
- Buscar un usuario por su identificador mediante GET.
- Actualizar los datos de un usuario mediante PUT.
- Eliminar usuarios mediante DELETE.

Estas operaciones permiten aplicar los principios de comunicación cliente-servidor explicados en el material de estudio.

4. Registro de trazabilidad de las consultas realizadas

La siguiente tabla recoge las interacciones principales de la conversación utilizada para desarrollar el proyecto. Los mensajes se presentan según su secuencia real, diferenciando las solicitudes de ayuda de los mensajes que contienen errores técnicos.

N.º	Prompt utilizado (versión formalizada)	Objetivo de la consulta	Información obtenida (palabras clave)	¿Cómo llevó a la siguiente consulta?
1	A partir del material de estudio sobre estructura HTTP y JSON, orienta la creación de una API REST con NestJS que permita gestionar usuarios mediante operaciones CRUD. Explica cómo organizar el proyecto y utilizar una base de datos local integrada, sin depender de una aplicación externa para administrarla.	Comprender cómo aplicar los conceptos de HTTP, JSON y API REST en el desarrollo de un backend con NestJS.	NestJS, API REST, HTTP, JSON, CRUD, SQLite, TypeORM, persistencia local.	Durante la configuración del proyecto surgieron errores relacionados con las dependencias y el tipo de conexión de la base de datos. Esto hizo necesario consultar cómo identificarlos y corregirlos.
2	Analiza los errores de compilación relacionados con el módulo `@nestjs/mapped-types` y el tipo de conexión de SQLite en TypeORM. Explica sus posibles causas y los pasos necesarios para corregirlos.	Identificar los problemas de configuración que impedían compilar correctamente el proyecto.	TypeScript, dependencias, `@nestjs/mapped-types`, TypeORM, `better-sqlite3`, configuración.	Al persistir un error en la configuración de `app.module.ts`, fue necesario revisar específicamente el archivo y comprender qué modificación debía realizarse.
3	Examina el error de TypeScript presentado en `src/app.module.ts`. Explica la incompatibilidad del tipo de base de datos y orienta la modificación de la configuración para continuar con el desarrollo de la aplicación.	Profundizar en la solución del error de configuración y conseguir que la aplicación pudiera iniciarse.	`app.module.ts`, tipos de conexión, configuración, compilación, diagnóstico de errores.	Después de avanzar con la aplicación, se decidió utilizar un array en memoria para gestionar los usuarios y simplificar la implementación. Posteriormente, fue necesario aprender a comprobar las operaciones mediante Postman.
4	Explica cómo probar una API REST desarrollada con NestJS utilizando Postman. Incluye ejemplos de solicitudes HTTP con los métodos GET, POST, PUT y DELETE, el envío de datos JSON y la comprobación de las respuestas del servidor.	Aprender a verificar los endpoints y comprobar las operaciones CRUD de la API.	Postman, endpoints, GET, POST, PUT, DELETE, JSON, códigos de estado HTTP, pruebas funcionales.	Las pruebas permiten comprobar el funcionamiento de las operaciones implementadas y determinar si el proyecto está listo para su compilación final

5. Evolución del proceso de desarrollo

5.1. Planteamiento inicial

El proceso comenzó con la necesidad de desarrollar una aplicación backend basada en los conceptos de HTTP y JSON, pero con la condición de mantener la base de datos dentro del propio proyecto.

A partir de esta necesidad, se planteó utilizar NestJS para construir la API REST y SQLite para almacenar los registros localmente.

5.2. Estructuración de la aplicación

La solución propuesta se organizó mediante módulos, controladores, servicios, entidades y DTO. Esta estructura permite separar la recepción de las solicitudes, la lógica de procesamiento, la validación de los datos y el acceso a la información almacenada.

También se definieron endpoints para gestionar usuarios mediante los métodos HTTP correspondientes.

5.3. Identificación y resolución de errores

Durante la configuración aparecieron dos problemas técnicos principales.

El primero fue la ausencia del paquete `@nestjs/mapped-types`, necesario para utilizar `PartialType` en el DTO de actualización.

El segundo fue la incompatibilidad del valor `sqlite` con los tipos de conexión admitidos por la versión de TypeORM instalada. Como alternativa, se propuso utilizar `better-sqlite3` e instalar sus dependencias.

Posteriormente, el error de compilación continuó apareciendo porque el archivo seguía mostrando `type: 'sqlite'`. Por ello, se indicó comprobar que el cambio se hubiera guardado correctamente.

Estas interacciones muestran cómo una consulta inicial puede conducir a una pregunta más específica cuando la solución propuesta no se refleja todavía en el código.

5.4. Preparación de las pruebas

Después de abordar los problemas de configuración, se solicitó orientación para probar la API con Postman.

Se explicó cómo seleccionar el método HTTP, introducir la URL correspondiente y enviar información en formato JSON. También se describieron las respuestas esperadas para las operaciones de creación, consulta, actualización y eliminación.

La comprobación definitiva depende de ejecutar las peticiones y revisar los resultados reales obtenidos en el entorno local.

6. Relación con los resultados de aprendizaje de la asignatura

El proyecto se relaciona con los resultados de aprendizaje establecidos en el programa de estudios de la siguiente manera:

Arquitectura web y desarrollo backend. Se aplican los conceptos de cliente-servidor mediante una API desarrollada en NestJS, organizada por módulos y componentes.

Desarrollo de servicios mediante API REST. Se utilizan endpoints y métodos HTTP para gestionar los recursos de usuarios, así como JSON para intercambiar información.

Gestión, validación y transferencia de datos. Se plantean DTO para validar los datos recibidos y TypeORM para realizar operaciones sobre la base de datos local.

Pruebas y verificación. Se propone utilizar Postman para comprobar las respuestas de los endpoints y detectar errores de funcionamiento.

El proyecto constituye una práctica inicial de estos contenidos. No se considera que incluya todas las competencias de la asignatura, como autenticación mediante tokens, autorización por roles, pruebas unitarias, transacciones o despliegue en Internet, ya que no forman parte de la implementación descrita hasta el momento.

7. Verificación de la información obtenida mediante IA

La información generada por ChatGPT debe revisarse antes de incorporarla al proyecto. En este caso, la verificación debe incluir los siguientes aspectos:

1. Comprobar que las dependencias estén instaladas correctamente.
2. Revisar que el tipo de conexión configurado sea compatible con la versión instalada de TypeORM.
3. Ejecutar la aplicación y comprobar que NestJS inicie sin errores.
4. Realizar peticiones HTTP en Postman y verificar los códigos de estado y las respuestas JSON.
5. Comprobar que los datos se conserven en el archivo SQLite después de reiniciar la aplicación.
6. Contrastar las configuraciones con la documentación oficial de NestJS, TypeORM y SQLite.

Las fuentes técnicas oficiales recomendadas para esta verificación son:

- NestJS: https://docs.nestjs.com/
- TypeORM: https://typeorm.io/
- SQLite: https://www.sqlite.org/docs.html

Estas páginas se incluyen como fuentes propuestas para contrastar la información técnica; su consulta y verificación deben realizarse antes de afirmar que se utilizaron como fuentes comprobadas.

8. Uso ético y responsable de la inteligencia artificial

El uso de ChatGPT se orientó a obtener explicaciones, ejemplos de código y asistencia para solucionar dificultades técnicas. Sin embargo, la utilización de esta herramienta no elimina la responsabilidad del estudiante sobre el código incorporado al proyecto.

De acuerdo con el apartado 12 del programa de la asignatura, el uso de inteligencia artificial debe declararse y los resultados obtenidos deben revisarse y verificarse antes de presentarse académicamente.

Por este motivo, el estudiante debe identificar qué partes fueron propuestas con ayuda de ChatGPT, comprender su funcionamiento, corregir los errores encontrados y verificar los resultados obtenidos mediante pruebas.

El presente informe constituye una declaración del uso de ChatGPT en el proceso descrito. No implica que se hayan completado verificaciones técnicas que todavía estén pendientes.

9. Resultados y limitaciones

El proceso permitió establecer una propuesta de estructura para una API REST de usuarios, definir las operaciones CRUD y obtener orientación para configurar la persistencia local mediante SQLite.

También se identificaron dificultades relacionadas con las dependencias y la compatibilidad del controlador de base de datos. Las consultas posteriores permitieron precisar el origen del error de compilación y preparar un procedimiento de pruebas mediante Postman.

No obstante, la funcionalidad completa debe demostrarse con evidencia del entorno de ejecución. Para ello, se recomienda adjuntar capturas de la aplicación iniciada, las peticiones realizadas en Postman, las respuestas obtenidas y la existencia del archivo de base de datos local.

10. Conclusión

La utilización de ChatGPT permitió avanzar de manera progresiva en el desarrollo de una API REST con NestJS, desde la definición de la solución hasta la preparación de las pruebas de los endpoints.

La secuencia de consultas evidencia un proceso de aprendizaje en el que las dificultades encontradas generaron nuevas preguntas y orientaron la búsqueda de soluciones más específicas.

El proyecto permite aplicar los fundamentos de HTTP, JSON, arquitectura cliente-servidor y persistencia de datos estudiados en la asignatura. Asimismo, demuestra la importancia de utilizar la inteligencia artificial de manera transparente, crítica y responsable.

Finalmente, se reconoce que la calidad de una aplicación no depende únicamente de obtener código mediante una herramienta de IA, sino de comprenderlo, comprobar su funcionamiento y corregir los problemas que puedan surgir durante su implementación.

https://github.com/daguzman-netizen/BDD_Local_Momento_1

